Course
TOPIC NAME : Software Engineering DAY : Sunday
TIME : DATE : 11 / 10 / 26

```
import java.util.concurrent.atomic.AtomicLong;
class CounterTask implements Runnable {
    private final int increments;
    private final boolean threadSafe;
private long nonStaticCounter = 0;
private static AtomicLong atomicStaticCou-
    nter = new AtomicLong(0);
private static long unsafeStaticCounter = 0;
public CounterTask(int increments, boolean
 threadSafe) {
    this.increments = increments;
    this.threadSafe = threadSafe;
}
@Override
public void run() {
for(int i=0; i < increments; i++) {
    nonStaticCounter++;
```

JT24022 (2)

TOPIC NAME _____ DAY _____

TIME _____ DATE / /

```
if (threadSafe) {
    atomicStaticCounter.incrementAndGet();
}
else {
    unsafeStaticCounter++;
}
}
}
public long getNonStaticCounter() {
    return nonStaticCounter;
}
public static long getStaticCounter
        (boolean threadSafe) {
if (threadSafe) {
return atomicStaticCounter.get();
}
```

GOOD LUCK™

```
    else {
      return unSafeStaticCounter;
    }
  }

  public static void resetCounters() {
    atomicStaticCounter.set(0);
    unSafeStaticCounter = 0;
  }
}

public class John_Thread {
  public static void main(String[] args)

if (args.length < 3) {
System.out.println("Usage: java John_
Thread <threads> <increments_per_thread>
  <true/false>");
    return;
}
```

JT24022 (4)

TOPIC NAME DAY TIME DATE / /

```
int numThreads = Integer.parseInt(args[0]);
int incrementsPerThread = Integer.
        parseInt(args[1]);
boolean threadSafe = Boolean.parseBo
        olean(args[2]);
CounterTask.resetCounters();
Thread[] threads = new Thread[numT
        hreads];
CounterTask[] tasks = new
        CounterTask[numThreads];
for (int i=0; i<numThreads; i++) {
    try {
        threads[i].join();
    }
```

```
        catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

long totalNonStatic = 0;
for (int i = 0; i < numThreads; i++) {
    totalNonStatic += tasks[i].getNonStatic
                    : counter.();
    }
long finalStatic = CounterTasks.
        getStaticCounter(threadSafe);
long expected = (long) numThreads *
        incrementsPerThread;
long absoluteDiff = Math.abs(finalStatic
        - totalNonStatic);
```

TOPIC NAME _____ DAY _____

TIME DATE / /

```
double percentageDiff = (totalNonStatic == 0)?
    :((double) absoluteDiff / totalNonStatic) * 100.0;

System.out.println("--- Experiment Results ---");
System.out.println("Threads (N): " +
                   numThreads);
System.out.println("Increments per
  thread (K): " + incrementsPerThread);
System.out.println("Mode (Thread-safe/
  Atomic): " + threadSafe);
System.out.println("Expected Count: "
  + expected);
System.out.println("Static Count (S): "
  + finalStatic);
```

JT24022 (7)

TOPIC NAME DAY

TIME DATE / /

```
System.out.println("Non-Static Total(I): "
        + totalNonStatic);

System.out.println("Absolute Difference
    (D): " + absoluteDiff);

System.out.println("Percentage Difference
    (P): " + percentageDiff + "%");
    }
}
```

JT24022 (8)

TOPIC NAME : DAY :

TIME DATE : / /

Result Recording Sheet :

Threads	Expected Count	Static Count (unsafe)	Non-static total	Absolute difference	Difference %
1	1000	1000	1000	0	0.0%
2	20000	10016	20000	9984	49.92%
5	50000	33854	50000	16146	32.29%
10	500000	283158	500000	216842	43.37%
20	1000000	631958	1000000	368042	36.80%
50	2500000	2119630	2500000	380370	15.21%
100	5000000	4464000	5000000	536000	10.72%